PROJECT REPORT

on

Go Pro

from

Techpile Technology Pvt. Ltd., Lucknow

Towards partial fulfillment of the requirements

for the award of degree of

Master of Computer Applications

from

Babu Banarasi Das University

Lucknow

Academic Session 2025 - 26

School of Computer Applications

I Floor, CA- BLOCK, BBD University, BBD City, Ayodhya Road, Lucknow (U. P.) INDIA 226028
PHONE: HEAD: 0522-6196242 **Dept. T&P Cell:** 9990462250

w w w . b b d u . a c . i n

PROJECT REPORT

on

Go Pro

from

Techpile Technology Pvt. Ltd., Lucknow

Towards partial fulfillment of the requirements for the award of degree of

Master of Computer Applications

from

Babu Banarasi Das University Lucknow

Developed and Submitted by
Tanmay Singh Bangari
1240212124

Under Guidance of
Mrs. Devika Singh
Assistant Professor

Academic Session 2025 - 26

School of Computer Applications

UNDER TAKING

This is to certify that Project Report entitled

Go Pro

being submitted by

Tanmay Singh Bangari

Towards the partial fulfillment of the requirements
for the award of the degree of

Master of Computer Applications

from

Babu Banarasi Das University
Lucknow

Academic Year 2025-26

is a record of the student's own work carried out at

Techpile Technology Pvt. Ltd., Lucknow

and to the best of our knowledge the work reported herein does not form a part of any other thesis or work on the basis of which degree or award was conferred on an earlier occasion to this or any other candidate.

Authorized Signatory
Techpile Technology Pvt. Ltd.

Students Signature
Tanmay Singh Bangari
1240212124

PROJECT COMPLETION CERTIFICATE

Enroll No: TechpileWT250029

This is to certify that **Tanmay Singh Bangari,** a student **Master of Computer Application** from **Babu Banarasi Das University**, has successfully completed his project titled **Go Pro** at **Techpile Technology Pvt. Ltd., Lucknow**.

The project was developed using **Java Spring Boot** during the internship period from **15th Jan to 25th April**. During this tenure, **Tanmay Singh Bangari** demonstrated commendable dedication, technical proficiency, and a strong willingness to learn. He actively contributed to the successful completion of the project and showed good understanding of development practices in the **Software Development & Testing Division**.

His overall performance throughout the training period has been found to be **Good**.

We appreciate his efforts and wish him all the best for future career endeavors.

DIVYA RAI
Project Manager
Techpile Technology Pvt. Ltd.

DECLARATION

I, hereby declare that the report entitled **"Go Pro"**, submitted to the **School of Computer Applications**, **Babu Banarasi Das University***, BBD City, Ayodhya Road, Lucknow, Uttar Pradesh – 226028*, is submitted in partial fulfilment of the requirements for the award of the degree of M**asters of Computer Applications.**

This report is the outcome of my own effort and has been completed under the guidance and supervision of **Mrs. Devika Singh**. I also declare that the contents of this report have not been submitted, either in full or in part, to any other university or institution for the award of any degree or diploma.

Place: Lucknow
Date:

Signature of the Student

Students' Name: Tanmay Singh Bangari
University Roll No.: 1240212124

Acknowledgement

It gives us immense pleasure to express our sincere gratitude to all those who extended their support and guidance throughout the course of this project. The successful completion of this work would not have been possible without the encouragement and assistance of many individuals.

First and foremost, we would like to extend our profound gratitude to **Dr. Reena Srivastava**, Honourable *Dean*, School of Computer Applications, Babu Banarasi Das University, for her continued support, motivation, and encouragement. Her visionary leadership and guidance have always been a source of inspiration for us. We are also deeply thankful to **Dr. Prabhash Chandra Pathak**, *Head of the Department*, Computer Applications, for his constant support and valuable insights, which played a vital role in shaping our project and helping us stay focused on our objectives.

We extend our deep appreciation to our project guide, **Mrs. Devika Singh**, for her unwavering support, expert guidance, and timely feedback throughout the duration of the project. Her willingness to assist us at every step and address our queries with patience greatly contributed to the successful completion of this work. We would also like to express our gratitude to the **Training and Placement Coordinators** for their support, cooperation, and guidance during the course of this project.

Last but not the least, we sincerely thank our **families and friends** for their constant support, understanding, and motivation. Their encouragement has been a strong pillar behind our consistent efforts.

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1.	Introduction	1-4
2.	Objective	5-7
3.	Purpose & Scope	8-10
4.	Survey of Technology	11-14
5.	Requirement Analysis	15-19
•	Problem Definition	15
•	Planning & Scheduling	15
•	Requirement Specification	15-18
•	SDLC, System Analysis, Feasibility	18-19
6.	Modules Contributor Responsibility & Modules	20-24
7.	Diagrams & Data Dictionary	25-31
8.	Coding	32-38
9.	System Design	39-45
10.	Testing	46-51
11.	Conclusion	52-53
12.	Future Scope	54-56
13.	Bibliography	57

CHAPTER 1: INTRODUCTION

1.1. Background of the Study

In recent times, urban transport has witnessed major changes in the advent of technology. Traditional methods of booking transportation such as street hailing and phone-based taxi services, have proved to be inefficient, unreliable, and wastage of time in most cases. Due to fast pace of urbanization and increase in population in urban areas, there has been a demand for a reliable, efficient, and affordable means of transporting people from place to place.

Ride-hailing platform applications have become the answer to all these problems. Ride-hailing applications such as Ola and Uber have changed the face of urban transport with their unique system where people can easily book rides through smartphones or computers. These platforms use technologies such as Global Positioning System (GPS), real-time data processing, and digital payment options to make the entire process smooth for the users.

The Go Pro Rental Vehicle Platform is the implementation of a ride-hailing platform that acts as a simulation of a transportation app. This application tries to create an effective solution by showing the working of the ride-hailing technology. The main focus of the project will be on ride booking, finding the right driver, calculating fares, and managing payments through object-oriented programming techniques.

The Go Pro Ride-Hailing Application is a software application used by users to hire drivers from a central application.

1.2. Introduction to the Project

The Go Pro system is a software application used for connecting users with drivers. This application system allows users to book rides, locate cars in real time, and complete payments digitally. The drivers, on the other hand, can accept ride booking requests, modify their status, and manage.

This application is coded using Java language, and a modular architecture is adopted. Each functionality is developed as a separate module. The main functionalities of this system are:

- Registration and login for users and drivers
- Driver registration and verification
- Vehicle management
- Booking a ride and confirmation
- Determining driver availability based on the trip location
- Calculating the total fare based on the distance and type of ride
- Transaction processing
- Maintaining wallets
- Tracking rides in real-time
- Allowing users to rate and review drivers
- Maintaining ride history

- Creating and managing notifications
- Allowing the use of promotional codes and discount coupons
- Providing an SOS button in emergencies
- Having a dashboard for administrative use

Collectively, all functionalities mentioned above simulate a real-world ride-sharing platform.

1.3. Need for the System

It is essential to have a highly efficient transport system owing to the shortcomings in the traditional approaches. The main challenges associated with traditional transport systems include:

- Vehicle availability identification
- Price transparency
- Ensuring safety measures
- Communication between the driver and rider
- Time delays and unpredictability

The Go Pro platform offers solutions to the above-mentioned problems in terms of:

- Quick ride booking through the digital medium
- Calculation of transparent fares
- Real-time tracking for safety and convenience reasons
- Automated driver allocation
- Secure and multiple payment options

The process is made efficient through digitization which helps avoid any kind of human error.

1.4. Overview of Ride-Hailing Systems

In a client-server model of ride hailing services, a user connects to the server via the frontend and makes his request which is managed by the backend server processing requests, managing data, and coordinating services.

The typical ride-hailing service workflow is as follows:

1. The user provides the pickup and destination location
2. The system calculates estimated fare and distance
3. Nearby drivers are identified
4. The ride request is accepting by a driver
5. The ride begins with real-time tracking
6. The ride is completed and payment is processed

Such a system is highly dependent on:

- The use of GPS for location tracking
- Cloud computing for scaling purposes
- APIs for communication
- Secure payment gateways

The Go Pro project is implemented by replicating the above-mentioned workflow in a simple and efficient manner using a simplified yet effective approach.

1.5. Scope of the Project

The scope of the Go Pro Ride-hailing simulated system involves the design and development of an entirely ride-hailing simulation system. The project entails both user-side and driver-side capabilities along with administrative control.

The system scope includes:

- Taking bookings and managing rides
- User and driver matching process
- Calculating fares dynamically
- Payment and wallet transactions management
- Providing ride tracking functionality
- To ensure the user safety features like offering an SOS emergency call feature
- Maintaining ride history tracks and analysis

The system does not include:

- Integration of real-time GPS services
- Live payment gateway integration
- Running on cloud-based infrastructure

Despite these constraints, the system effectively succeeds in demonstrating the core working mechanism of ride-hailing applications in the current world.

1.6. Objectives of the Project

The fundamental goal of developing Go Pro system is to create and develop an efficient and modular ride-hailing application that pretends real-world conveyance facilities.

The specific objectives are:

- The design of a convenient user-friendly interface for booking rides
- To implement efficient driver matching algorithms
- To develop a fare calculation system
- To integrate secure payment processing
- To implement safety features like SOS emergency call
- To ensure a scalable and maintainable system architecture

1.7. Significance of the Project

This project holds great importance and is significant in both academic as well as practical outlooks.

Academic Significance**:**

- Shows implementation and demonstration of Object-Oriented Programming (OOP) concepts
- Provides practical knowledge of system designing and developing systems
- Enhances and empowers knowledge regarding real-world software systems

Practical Significance:

- Creates a simulates of a real-life ride-hailing platform system
- Bridges gaps regarding transportation system problems
- Paves the way for developing a real-world practical application system

1.8. Organization of the Report

This report is structured in a way by dividing into multiple chapters to facilitate a systematic understanding of the project:

- **Chapter 1:** Introduction to the project
- **Chapter 2:** Objectives and goals
- **Chapter 3:** Purpose and scope
- **Chapter 4:** Survey of technologies
- **Chapter 5:** Requirement analysis
- **Chapter 6:** Modules and system design
- **Chapter 7:** Diagrams and data dictionary
- **Chapter 8:** Coding and implementation
- **Chapter 9:** Design screenshots
- **Chapter 10:** Testing
- **Chapter 11:** Conclusion
- **Chapter 12:** Future scope
- **Chapter 13:** Bibliography

1.9. SUMMARY

In this chapter, we have briefly introduced the concept of the Go Pro Rental Vehicle Platform and its importance in overcoming the issues associated with urban transportation. This chapter also sheds light on the background, necessity, objectives, and scope of the project. Furthermore, it gives a brief insight into working of system along with its emphasis on the significance of the project from both academic and practical perspectives.

Overall, this chapter establishes a strong foundation for understanding the system and sets the stage for the detailed discussions in subsequent chapters.

CHAPTER 2: OBJECTIVE

2.1. Introduction

The purpose of any software system refers to what is intended by developing the software and what can be achieved through that software. In case of Go Pro Rental Vehicle Platform, the purpose behind designing the system is to design an end-to-end ride-hailing platform that works on real-life ride-hailing just like Ola and Uber platform applications.

This chapter outlines and covers both the primary and specific purposes of the Go Pro Rental Vehicle Platform system, focusing entirely on functionality, performance, and user experience.

2.2. Primary Objective

The primary objective of the Go Pro system is:

To create a scalable, efficient and modifiable ride-hailing software platform that allows users to make booking of rides, contact with drivers, and undertake journeys through a web interface.

This system is designed and illustrate to demonstrate the use of object-oriented programming techniques for booking rides, allocation of drivers, calculation of fare, and payment processing.

2.3. Specific Objectives

In order to achieve and accomplish the above stated objective, the following are the specific objectives for the development of the project as follows:

2.3.1. User Management

- To implement user registration and validation
- To allow users to log in and manage their profiles
- To ensure secure user access using verification mechanisms

2.3.2. Driver Management

- To enable driver registration and verification
- To allow the status of drivers to go online/offline
- To maintain driver availability and ratings

2.3.3. Ride Booking System

- The ability of users to book rides by entering pickup and destination locations
- To generate ride requests dynamically
- To track and manage ride status (requested, confirmed, completed)

2.3.4. Ride Matching

- To identify and assign the availability of nearest driver
- To optimize driver selection based on availability and rating

2.3.5. Fare Calculation

- To determine ride fare depending on distance and vehicle type
- To implement surge fare pricing when required
- To estimate the fare before booking the ride

2.3.6. Payment System

- To offer and support various payment options such as UPI, cash, and wallet
- To process payments securely
- To handle payment transaction either success or failure, and refund if failed transaction

2.3.7. Wallet Management

- To add and manage funds in the wallet
- To keep the track of transaction history
- To support wallet-based payments

2.3.8. Ride Tracking

- To simulate real-time ride tracking
- To display driver's live location and estimated arrival time
- To monitor rider's progress track

2.3.9. Rating and Review System

- To allow passengers to rate drivers
- To keep and maintain track of average ratings
- To improve the service quality provided

2.3.10. Notification System

- To notify and update the users about their ride's status
- To send notification alerts for payments and promotions
- To maintain proper communication between system and users

2.3.11. Promo Code System

- To provide offer discounts on fares using promo codes
- To manage the validity of the promos
- To enhance user engagement

2.3.12. Emergency SOS Feature

- To assure that the user remains safe while being in the ride
- To allow sending out emergency alerts

- To notify emergency contacts

2.3.13. Admin Dashboard

- To monitor the activity in the system
- To manage users and drivers of the system
- To generate and create reports and analytics

2.4. Technical Objectives

The following technical objectives should be achieved by the system:

- To implement Object-Oriented Programming (OOP) principles such as encapsulation, inheritance, and polymorphism
- To create and design a modular architecture consisting of independent modules
- To make the software reusable and maintainable
- To simulate real-time processes effectively
- To develop and design a user-friendly graphical interface based on Java Swing technology

2.5. Performance Objectives

- To provide fast and efficient ride bookings
- To reduce the response time of the system
- To process the requests of multiple users and drivers simultaneously
- To ensure system reliability and stability

2.6. Security Objectives

- To ensure secure user authentication
- To protect the confidentiality of user data and transactions
- To prevent any unauthorized access
- To maintain and preserve data integrity

2.7. SUMMARY

The objectives of Go Pro Rental Vehicle Platform have been described in this chapter starting from the general objective of creating an integrated ride-hailing system. This chapter covered the specific objectives related to functionality, technical design, performance, and security.

Each objective is supported by the corresponding module of the system discussed in the next chapters.

CHAPTER 3: PURPOSE & SCOPE

3.1. Introduction

Each and every software system is developed with a specific purpose and a defined scope. The purpose defines the reason for creating a system, whereas the scope refers to the boundaries of the functionalities of the system. In terms of the Go Pro Rental Vehicle Platform, the purpose refers to the rationale for developing a structured, systematic way of handling ride-hailing services, whereas the scope refers to the features and limits of the system.

This chapter provides information about the purpose, application, and the applicability of the system to real-world scenarios.

3.2. Purpose of the System

The main objective of creating the Go Pro platform was the development of a ride-hailing software system that would facilitate interaction between the user and driver through a computer or web interface. The system has been created and designed to simulate real-world transportation services and showcase how ride-sharing application platforms operate in practise.

The key objectives of the applications include:

- To provide and supply an efficient method for booking transportation rides
- To reduce dependence on traditional public transports
- To ensure transparency and clarity in prices and operations
- To erect user safety wall through tracking and emergency options
- To create and provide a centralized platform for managing users, drivers, and rides

The main goal is to simplify the process of arranging transportation easier and automating booking, tracking, and payment processes.

3.3. Scope of the System

The scope of the Go Pro Rental Vehicle Platform defines the features and a range of functionalities that the system covers. It includes both user-side and administrative features.

3.3.1. Functional Scope

The system will have the following capabilities:

User-Side Features

- User registration and login
- Ride booking with pickup point and destination input
- Fare estimation before booking
- Real-time ride tracking
- Payments with different payment methods
- Viewing ride history
- Evaluation and rating of the driver

Driver-Side Features

- Driver registration and verification
- Availability management (online/offline)
- Request acceptance or rejection of ride requests
- Viewing ride information details
- Updating ride status

System Features

- Driver matching based on availability
- Dynamic fare calculation
- Notification system for updates
- Promo code and discount system
- Wallet management system
- Emergency SOS feature

Admin Features

- Monitoring the users and drivers
- Managing and monitoring trips and transactions
- Viewing system statistics and reports

3.4. Operational Scope

The system will run and operate in a simulated environment using Java-based technologies. The system does not require real-time deployment as it simulates how a real-world system would function in real-life situations.

The operational scope includes:

- Console and GUI-based interaction interface
- Simulation of ride booking and tracking
- Handling of multiple users and drivers
- Execution of system modules independently

3.5. Limitations of the System

Though the Go Pro platform considers and demonstrates a comprehensive and wide range of system functionalities, it has several limitations:

- Simulated real-time GPS tracking without connecting to actual maps
- Simulated payment processing using real payment gateways
- Lack of a cloud computing infrastructure platform
- System cannot scale beyond the local machine system
- No real-time database connectivity (extension is optional)

These limitations exist because of the fact that the system is developed and implemented for sake of academic purposes only. It focuses on conceptual understanding rather than full-scale deployment.

3.6. Application Areas

The Go Pro Rental Vehicle Platform system can be utilized in numerous domains, including:

- Urban transportation systems
- Ride-sharing platform application
- Logistics and transportation services
- Fleet management systems

Also, the system serves as a foundation for developing commercial solutions for ride-hailing applications.

3.7. Advantages of the System

- Creates and provides a structured and automated transportation framework
- Reduces efforts and time consumption in manual operations
- Enhances user convenience and safety
- Supports multiple functionalities in a single platform
- Demonstrates real-world system design

3.8. SUMMARY

In this chapter, we considered the purpose and scope of the Go Pro Rental Vehicle Platform. It analysed and highlighted the main objectives behind developing the system and described the functionalities it supports. Furthermore, this chapter also outlined the operational boundaries, limitations, and application use case areas of the system.

Overall, the scope of the platform guarantees that the system implements all essential aspects needed for a ride-hailing platform while remaining suitable for academic implementation. This sets the foundational basis for further understanding the technologies and requirements discussed in the next chapters.

CHAPTER 4: SURVEY OF TECHNOLOGY

4.1. Introduction

The construction and development of modern-day software system require the integration of numerous technologies to be incorporated within them. The Go Pro Rental Vehicle Platform is built using a combination of programming concepts, development tools, and system design methodologies that enable efficient implementation of a ride-hailing system framework.

This chapter outlines the technologies used in the project under consideration, along with a brief discussion of technologies used in actual ride-hailing platforms. This section gives more insight about the technologies behind the development of the chosen software and how that software relates to industry standards.

4.2. Programming Language: Java

The main programming language employed in developing the Go Pro platform system is Java. Java is one of the most popular high-level, object-oriented programming language widely used for building scalable and secure software application products.

Key Features of Java:

- Platform independent (Write Once, Run Anywhere)
- Object-Oriented (encapsulation, inheritance, polymorphism)
- Robust and secure
- Rich standard library
- Upholds strong memory management

Role in the Project:

All components of the Go Pro Ride Hailing Platform are used to implement all modules including user management, ride booking, payments processing, and tracking. The modular structure of the system is based on object-oriented principles.

4.3. Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming paradigm based on object-oriented programming concepts that revolves around objects and classes.

Key OOP Concepts Used:

- **Encapsulation:** Wrapping up of data and methods into a single unit shows encapsulation (e.g., User, Driver classes)
- **Inheritance:** The process of inheriting properties of one class from another class enabling the reuse of code
- **Polymorphism:** Ability to take more than one for allowing methods to perform different tasks
- **Abstraction:** The act of representing essential features without going back to background details hiding complex implementation details.

Role in the Project:

All modules of the system have been created using the classes, including the following ones:

- User
- Driver
- Ride Booking
- Payment

This ensures making the system modular and maintainable.

4.4. Java Swing (GUI Technology)

Java Swing is employed to construct the graphical user interface (GUI) of the system. Swing provides a collection of components for developing interactive software.

Features of Swing:

- Platform-independent GUI
- Wide range of UI components (buttons, panels, tables)
- Customizable interface design

Role in the Project:

The system comprises a GUI dashboard with options where users can:

- Register and log in
- Arrange ride bookings
- View ride history
- Manage payments

4.5. Database Concept

Although currently using in-memory data structures (like lists), principles of database design have been applied in the system.

Common Databases Used in Industry:

- MySQL
- MongoDB
- PostgreSQL

Role in the Project:

- Stores information about users, drivers, and rides (simulating databases using collections)
- Can be extended to real database integration

4.6. GPS and Location Tracking

GPS (Global Positioning System) is a technology that determines the real-time location of devices.

Role in Ride-Hailing Systems:

- Tracks driver and user location
- Measures distances and calculate routes
- Provides estimated arrival time (ETA)

Role in the Project:

The system simulates GPS tracking by updating driver location and calculating distance within the system.

4.7. Payment Technologies

Nowadays, modern applications usually implement secure digital payment systems.

Common Payment Methods:

- UPI
- Credit/Debit Cards
- Digital Wallets

Role in the Project:

The system includes simulation of:

- Payment processing
- Payment status (success or if failed then refunded)
- Wallet transactions

4.8. API (Application Programming Interface)

APIs allow interaction and communication between different software modules.

Examples in Real Systems:

- Google Maps API (for navigation)
- Payment Gateway APIs
- Notification APIs

Role in the Project:

Even though APIs are not directly included within the framework, there is a way of designing which will support API integration in the future.

4.9. Cloud Computing

Through cloud computing applications run on remote servers and can providing scalability and availability depending on the requirement.

Popular Cloud Platforms:

- AWS (Amazon Web Services)
- Microsoft Azure
- Google Cloud Platform

Role in Ride-Hailing Systems:

- Serves as storage for huge amount of data
- Handles multiple users simultaneously
- Ensures high availability

Project Context:

The current system application runs locally but can be scaled to cloud deployment in the future.

4.10. Software Development Tools

The software development tools being used during the development process includes:

- **IDE (Integrated Development Environment):**
    - IntelliJ IDEA / Eclipse / NetBeans
- **Version Control (optional):**
    - Git
- **Documentation Tools:**
    - MS Word (for report preparation)

4.11. Comparison with Real-World Systems

Feature	Go Pro System	Real Systems (Ola/Uber)
Ride Booking	Yes	Yes
GPS Tracking	Simulated	Real-time
Payment	Simulated	Real Payment Gateway
Database	In-memory	Cloud Database
Scalability	Limited	High
Deployment	Local	Global

This comparison analysis shows that even though the project is simple, it greatly resembles real-world architecture.

4.12. SUMMARY

In this chapter, an overview of the technologies used in developing the Go Pro Ride Hailing System has been outlined. Java technology, Object-Oriented Programming, GUI development using Swing, as well as the ideas of GPS tracking, payment system services, and other API technologies, and cloud computing were explored.

In addition to this, the chapter also analysed and compared the project in relation to the current ride-hailing systems highlighting similarities and differences. In general, the technology behind this project is solid foundation for understanding and learning about software development and real-time transportation systems.

CHAPTER 5: REQUIREMENT ANALYSIS

5.1. Introduction

The requirement analysis is one of the most crucial phases of the Software Development Life Cycle (SDLC). The main focus of this stage is to determine, analyse, and documenting the needs and expectations of users and stakeholders. This phase ensures that the system is developed according to user requirements and functions effectively.

In the case of Go Pro Rental Vehicle Platform, requirement analysis is responsible for determining what the system should do, why and how the system will perform under certain limitations.

5.2. Types of Requirements

The requirements of the system may be broadly classified as:

- Functional Requirements
- System Requirements
- Non-Functional Requirements

5.3. Functional Requirements

Functional requirements refer to the basic functionalities provided by the system. These are derived directly from project modules.

5.3.1. User Management Requirements

- The system shall provide users to register with personal details
- The system shall support login verification
- The system must authenticate users via OTP

Reference Implementation:

5.3.2. Driver Management Requirements

- The system shall facilitate driver registration process
- The system must verify drivers based on license number details
- Drivers should be provided with an option to go online/offline

Reference Implementation:

5.3.3. Ride Booking Requirements

- Users must be allowed to book rides through provision of pickup and destination location
- The system must generate a unique ride ID
- The system must track ride booking status

Reference Implementation:

5.3.4. Ride Matching Requirements

- The system must identify the nearest available driver
- The system must auto-assign rides
- Driver selection must depend upon availability and rating

Reference Implementation:

5.3.5. Fare Calculation Requirements

- The system must calculate fare based upon distance and vehicle type
- The system must implement surge pricing
- Fare details should be displayed before ride booking confirmation

Reference Implementation:

5.3.6. Payment Requirements

- The system must enable multiple payment methods
- The system must facilitate safe payment processing
- The system must manage payment status (success, failed, refund)

Reference Implementation:

5.3.7. Wallet Management Requirements

- The system must facilitate adding money to wallet
- The system must track transactions
- The system must allow supporting wallet-based payments

Reference Implementation:

5.3.8. Ride Tracking Requirements

- The system must provide real-time ride tracking simulation
- The system must show distance covered and ETA details

Reference Implementation:

5.3.9. Notification Requirements

- The system must send ride updates to user
- The system must notify payment status to user
- The system must send promotional alert notifications

Reference Implementation:

5.3.10. Rating & Review Requirements

- The system must let users to rate drivers
- The system must calculate average ratings

Reference Implementation:

5.3.11. Promo Code Requirements

- The system must support promo and discount codes
- The system must validate usage and expiry

Reference Implementation:

5.3.12. SOS Emergency Requirements

- The system must allow users to trigger emergency alert calls
- The system must notify and inform emergency contacts

Reference Implementation:

5.3.13. Admin Dashboard Requirements

- The system must display statistics of the system
- The admin must manage the users and drivers
- The system must generate and produce reports

Reference Implementation:

5.4. Non-Functional Requirements

Non-functional requirements refer to the quality attributes and system performance.

5.4.1. Performance Requirements

- The system should respond quickly to user requests
- The ride booking request should be processed efficiently

5.4.2. Security Requirements

- User data must be protected
- Authentication process must be secure
- Any unauthorized access must be prevented

5.4.3. Reliability Requirements

- The system should function without failure
- Data consistency must be maintained

5.4.4. Scalability Requirements

- The system should handle multiple users
- The system shall allow scalability for future expansion

5.4.5. Usability Requirements

- The interface should be user-friendly
- The system should be easy to navigate

5.5. System Requirements

5.5.1. Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space

5.5.2. Software Requirements

- Operating System: Windows / Linux
- Programming Language: Java
- IDE: IntelliJ / Eclipse / NetBeans
- GUI: Java Swing

5.6. Feasibility Analysis

5.6.1. Technical Feasibility

The system is technically feasible as it incorporates standard technologies like Java and Swing.

5.6.2. Economic Feasibility

The project is economically feasible and cost-effective since it incorporates open-source tools.

5.6.3. Operational Feasibility

The system is easy to use and has operational feasibility without specialized training.

5.7. SUMMARY

This chapter analysed the requirements of the Go Pro Rental Vehicle Platform in detail. It covered functional requirements which were based on different system modules, non-functional requirements pertaining to performance and security, as well as system requirements for implementation.

The chapter also covered feasibility analysis to verify that the system is practical and realistic. The above requirements provide the foundation for system design and implementation discussed in the following chapters.

CHAPTER 6: MODULES & RESPONSIBILITY

6.1. Introduction

The Go Pro Rental Vehicle Platform is designed following a modular architecture, where the entire system is divided into multiple independent modules. Each module is responsible for performing a separate function independently ensuring better organization, maintainability, and scalability of the system.

This chapter explains each module in detail along with its responsibilities and role in the overall system.

6.2. Overview of System Modules

The following are the important modules present in the system:

1. User Module
2. Driver Module
3. Vehicle Module
4. Ride Booking Module
5. Ride Matching Module
6. Fare Calculation Module
7. Payment Module
8. Wallet Module
9. Ride Tracking Module
10. Rating & Review Module
11. Ride History Module
12. Notification Module
13. Promo Code Module
14. SOS Emergency Module
15. Admin Dashboard Module

All modules work together to complete the ride lifecycle process from beginning till end.

6.3. Module Descriptions and Responsibilities

6.3.1. User Module

Responsibilities:

- User registration and login
- OTP verification
- Profile management

Functions:

- Authenticate users
- Manage activities during sessions
- Store user details

Role in System:
Serves as the gateway for passengers using the platform.

6.3.2. Driver Module

Responsibilities:

- Driver registration and verification
- Availability management (online/offline)
- Maintaining driver rating

Functions:

- Verify license details
- Update driver availability status
- Track driver performance

Role in System:
Furnishes drivers for allocating rides to customers.

6.3.3. Vehicle Module

Responsibilities:

- Monitor vehicle details
- Associate vehicles with drivers
- Maintain vehicle status

Functions:

- Add/update vehicle information
- Activate/deactivate vehicles

Role in System:
Releases vehicle(s) that act as the transportation medium used in rides.

6.3.4. Ride Booking Module

Responsibilities:

- Create ride booking requests
- Control the lifecycle of ride booking
- Keep track of ride details

Functions:

- Generate ride ID
- Update ride status
- Handle ride confirmation

Role in System:
Core module that initiates the ride booking process.

6.3.5. Ride Matching Module

Responsibilities:

- Find suitable drivers
- Assign rides automatically

Functions:

- Monitors available drivers
- Select best driver based on rating

Role in System:
Allocates the right driver for an upcoming ride.

6.3.6. Fare Calculation Module

Responsibilities:

- Estimate ride fare
- Enable surge pricing

Functions:

- Compute fare based on distance
- Adjust fare dynamically

Role in System:
Provides ride fare transparency.

6.3.7. Payment Module

Responsibilities:

- Process customer payments
- Update payment status

Functions:

- Complete customer transactions
- Manage refunds

Role in System:
Ensures financial transactions between parties is completed.

6.3.8. Wallet Module

Responsibilities:

- Monitors user wallet
- Store transaction history

Functions:

- Add/deduct balance
- Track wallet usage

Role in System:
Offers a substitute payment technique.

6.3.9. Ride Tracking Module

Responsibilities:

- Track ride progress
- Estimate arrival time

Functions:

- Update driver location
- Calculate ETA

Role in System:
Enhance and improve user experience and safety.

6.3.10. Rating & Review Module

Responsibilities:

- Gather feedback
- Maintain ratings

Functions:

- Submit reviews
- Calculate average ratings

Role in System:
Improves quality of services provided.

6.3.11. Ride History Module

Responsibilities:

- Store information about past rides
- Provides ride statistics

Functions:

- Display ride records
- Calculate overall spending

Role in System:
Assists users in tracking previous rides.

6.3.12. Notification Module

Responsibilities:

- Send alerts and updates
- Maintain ride notification records

Functions:

- Notify ride status
- Sends payment updates

Role in System:
Guarantees good communication between system and users.

6.3.13. Promo Code Module

Responsibilities:

- Manages and maintains discounts
- Validates promo code

Functions:

- Applies discounts
- Tracks and monitors promocodes

Role in System:
Improves user experience with system offerings and promotions.

6.3.14. SOS Emergency Module

Responsibilities:

- Provides safety features
- Triggers emergency alerts

Functions:

- Sends emergency notification alerts
- Share user location

Role in System:
Guarantees user's safety during their rides.

6.3.15. Admin Dashboard Module

Responsibilities:

- Monitors track of system activities
- Manages users and drivers

Functions:

- Generate reports
- Provides statistics

Role in System:
Controls the whole system and supervises all actions taken.

6.4. Interaction Between Modules

All modules are interconnected and work together:

- User → Ride Booking → Ride Matching → Driver
- Fare Calculation → Payment → Wallet
- Ride Tracking → Notification → User

- Admin monitors all modules

This interaction ensures smooth execution of the ride lifecycle.

6.5. Advantages of Modular Design

- Easy to maintain and upgrade
- Improves code reusability
- Enhances scalability
- Simplifies debugging and testing easier

6.6. Team Members & Contribution

1. **Nihal Jamal:** I have contributed significantly to the development and design of the Go Pro Rental Vehicle Platform. His responsibilities included:

- Designing system architecture and overall workflow
- Developing core modules such as Ride Booking and Ride Matching
- Assisting in database structure and data flow design
- Creating diagrams including DFD and ER Diagram
- Participating in testing and debugging of the system

2. **Tanmay Singh Bangari:** I played a key role in implementation and documentation of the project. His contributions include:

- Developing modules such as User Management, Payment System, and Wallet
- Implementing GUI components and improving user interface
- Writing and compiling the project report
- Preparing diagrams such as Use Case, Class, and Sequence diagrams
- Performing testing, validation, and final integration of modules

6.7. SUMMARY

In this chapter the modular structure of the Go Pro Rental Vehicle Platform has been elucidated. A detailed explanation of each module, its functionalities and the roles in the system has been provided. The interaction between modules was also discussed, elaborating on how they collectively work together to complete the ride process.

The modular architecture design ensures that the system is efficient, scalable, and easy to manage. This structure forms the foundation for system design and development will be covered in the next chapter.

CHAPTER 7: DIAGRAMS & DATA DICTIONARY

7.1. Introduction

System design is an important phase in the software development process where the system illustrates and provides a visual and structural representation of how the system operates. Diagrams help in understanding system flow, data movement, and relationships between different components.

This chapter outlines important diagrams used in Go Pro Rental Vehicle Platform along with a detailed data dictionary describing the data elements used in the system.

7.2. Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) is used to show the flow of information within the system. The information shows inputs, processing of inputs, outputs, and data storage.

7.2.1. DFD Level 0 (Context Diagram)

The Level 0 DFD is used to represent the entire system as a single process.

External Entities:

- User
- Admin
- Driver

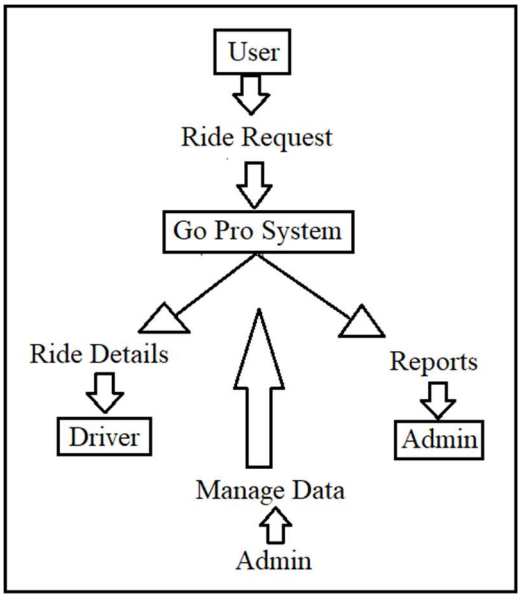
Main Process:

- Go Pro Ride-Hailing System

Data Flows:

- User sends ride request
- Driver receives request
- System processes request
- Admin monitors system

Explanation:
This diagram shows the general flow of system interactions between the system and the external entities.



7.2.2. DFD Level 1

The Level 1 DFD shows the major processes in the system:

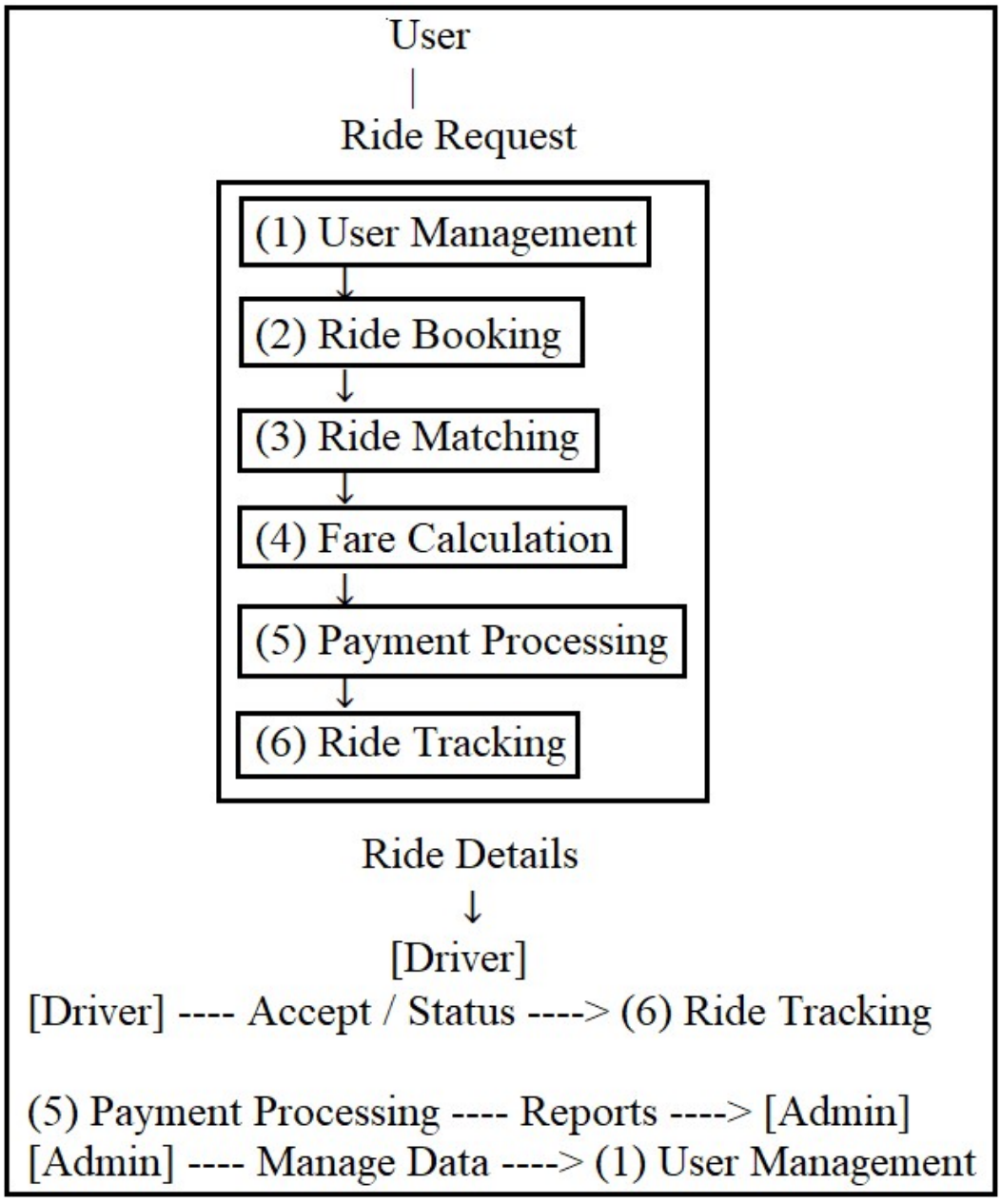
Processes:

1. User Management
2. Driver Management
3. Ride Booking
4. Payment Processing
5. Ride Tracking

Data Stores:

- User Database
- Driver Database
- Ride Database
- Payment Database

Explanation:
The above figure shows how data flows between different modules and databases.



7.3. Entity Relationship (ER) Diagram

The ER diagram explains the relationships between various entities within the system.

Entities:

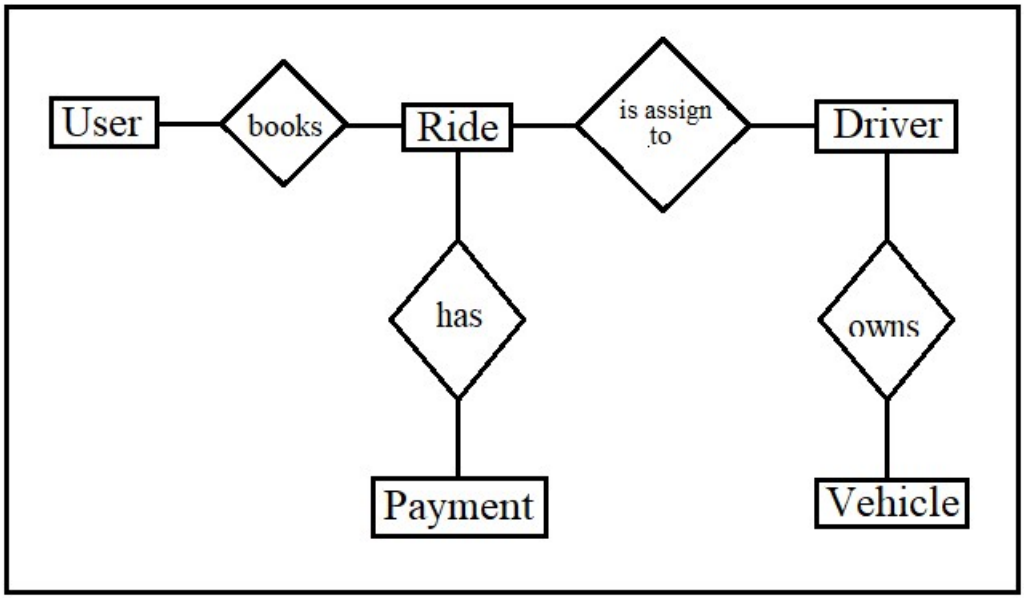
- User
- Driver
- Vehicle
- Ride
- Payment

Relationships:

- A user can book multiple rides
- A driver can handle multiple rides
- A ride is linked to one user and one driver
- A payment is linked to a ride

Explanation:

The ER diagram is used in designing the database structure and relationships among various entities within the database.



7.4. Use Case Diagram

The Use Case Diagram gives an illustration of use cases within the system and determines the interactions between users and the system.

Actors:

- User
- Driver
- Admin

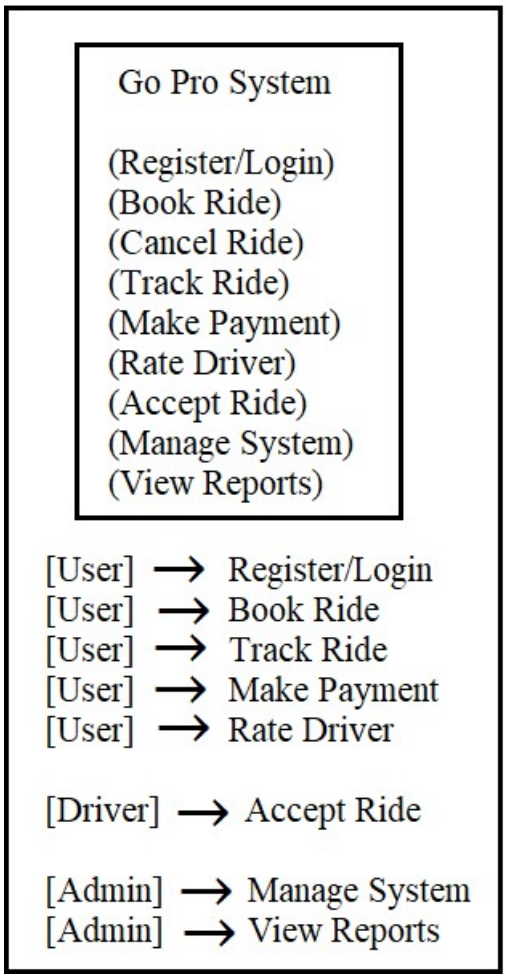
Use Cases:

- Register/Login
- Book Ride
- Accept Ride
- Make Payment

- Track Ride
- Managing the system

Explanation:

This use case diagram provides a functional overview of the system from the user's perspective.



7.5. Class Diagram

The Class Diagram represents the structure of the system in terms of classes and their relationships. The class diagram gives the illustration of objects in a class diagram.

Main Classes:

- User
- Driver
- Vehicle
- Ride Booking
- Payment
- Wallet
- Notification

Relationships:

- Association between User and Ride
- Association between Driver and Vehicle
- Dependency between Ride and Payment

Explanation:

This diagram indicates and reflects the object-oriented nature of the system.

7.6. Sequence Diagram

The Sequence Diagram shows the interaction of the system components over time.

Process Flow:

1. User requests ride
2. The system acts on user request
3. Driver is assigned
4. Ride starts
5. Payment is processed

Explanation:

It assists in understanding of the process and its operations in sequence within the system

Data Dictionary

The Data Dictionary is description of all the data elements used in the operation of the system.

7.6.1. User Table

Field	Type	Description
user_id	String	Unique user ID
name	String	User name
email	String	Email address
phone	String	Contact number
password	String	User password

7.6.2. Driver Table

Field	Type	Description
driver_id	String	Unique driver ID
name	String	Driver name
phone	String	Contact number
license_number	String	License details
rating	Double	Driver rating

7.6.3. Vehicle Table

Field	Type	Description
vehicle_id	String	Vehicle ID
driver_id	String	Associated driver
vehicle_type	String	Type of vehicle
number	String	Vehicle number

7.6.4. Ride Table

Field	Type	Description
ride_id	String	Unique ride ID
user_id	String	User reference
driver_id	String	Driver reference
pickup	String	Pickup location
destination	String	Destination
status	String	Ride status
fare	Double	Ride fare

7.6.5. Payment Table

Field	Type	Description
payment_id	String	Payment ID
ride_id	String	Ride reference
amount	Double	Payment amount
method	String	Payment type
status	String	Payment status

7.7. Importance of Diagrams

- Simplifies system understanding
- Helps in design and implementation process
- Improves communication among developers and designers
- Useful for documentation and presentation of system

7.8. SUMMARY

This chapter contained the design diagrams and data dictionary of the Go Pro Rental Vehicle Platform. The design diagrams include Data Flow Diagrams, ER diagrams, Use Case diagrams, Class diagrams, and Sequence diagrams to illustrate the system structure and operational flows.

The data dictionary offered an explanation of the data elements utilized within the system. The importance of these diagrams and data descriptions lies in their role for understanding architecture and serve as a blueprint for the system.

CHAPTER 8: CODING / IMPLEMENTATION

8.1. Introduction

The implementation phase involves translating the system design being converted into a real-time application that works with help of some programming language. In our case, the Go Pro Rental Vehicle Platform is designed using Java, adhering to the principles of object-oriented programming principles and a modular design approach.

Each functional part of the system has been coded as an independent class (module), allowing for better maintenance, reuse, and scalability. The system simulates a real-world ride-hailing application system by integrating different modules of the system such as ride booking, driver management, payment processing, and tracking.

8.2. System Architecture

A modular approach to the design of this software has been employed wherein each module is responsible to perform a particular function.

Main Components:

- User Interface (Console + GUI)
- Business Logic (Core modules)
- Data Handling (Collections)

Execution Flow:

1. User registers/logs in
2. Ride is requested
3. Driver is matched
4. Fare is calculated
5. Ride is completed
6. Payment is processed

The central execution is done through the main application class.

8.3. Main Application (Entry Point)

The system starts from the main class:

GoProApp.java

Responsibilities:

- Initializes all modules
- Demonstrates interaction between modules
- Simulates real system workflow

Key Operations:

- User and driver creation
- Ride booking and matching
- Fare calculation
- Payment processing

8.4. User Module Implementation

User.java

// shows authentication + registration logic

```
public boolean login(String email, String password) {
```

```
    return this.email.equals(email) && this.password.equals(password);
}
public void register(String name, String email, String phone) {
    this.name = name;
    this.email = email;
    this.phone = phone;
}
```

Key Features:

- User registration
- Login authentication
- OTP verification

Important Methods:

- login() → verifies user credentials
- verifyOTP() → validates user

Purpose:

Deals with all user-related operations and authentication.

8.5. Driver Module Implementation

Driver.java

```
// shows availability + rating logic
public void goOnline() {
    this.isAvailable = true;
}
public void goOffline() {
    this.isAvailable = false;
}
public void updateRating(double rating) {
    this.rating = (this.rating + rating) / 2;
}
```

Key Features:

- Driver verification
- Availability management
- Rating system

Important Methods:

- verifyDriver()
- goOnline() / goOffline()
- updateRating()

Purpose:

Manages driver lifecycle and availability.

8.6. Ride Booking Module

RideBooking.java

// shows ride lifecycle

```
public void confirmRide() {
   this.status = "CONFIRMED";
}
public void startRide() {
   this.status = "ONGOING";
}
public void completeRide() {
   this.status = "COMPLETED";
}
```

Key Features:

- Ride creation
- Lifecycle management of rides
- Status management

Important Methods:

- confirmRide()
- completeRide()
- startRide()

Purpose:

Acts as the core component managing rides.

8.7. Ride Matching Module

RideMatching.java

// core matching logic

```
public Driver findDriver(List<Driver> drivers) {
   for (Driver d : drivers) {
      if (d.isAvailable()) {
         return d;
      }
   }
   return null;
}
```

Key Features:

- Driver selection
- Matching based on availability

Logic Used:

- Selects driver with highest rating among available drivers

Purpose:

Ensures efficient allocation of drivers to users.

8.8. Fare Calculation Module

Fare = (Distance × Rate) × Surge Multiplier

//fare formula implementation

```
public double calculateFare(double distance, double rate, double surge) {
   return (distance * rate) * surge;
}
```

Key Features:

- Supports multiple vehicle types
- Surge pricing
- Dynamic pricing

Purpose:

Calculates cost of the ride accurately.

8.9. Payment Module

Payment.java

//shows validation+ status handling

```
public boolean processPayment(double amount) {
   if (amount > 0) {
      this.status = "SUCCESS";
      return true;
   }
   this.status = "FAILED";
   return false;
}
```

Key Features:

- Payment processing
- Status handling (Success, Failed, Refund)

Important Methods:

- processPayment()
- refundPayment()

Purpose:

Handles all financial transactions.

8.10. Wallet Module

Wallet.java

// real-world wallet logic

```
public void addMoney(double amount) {
   this.balance += amount;
}
public boolean deductMoney(double amount) {
   if (balance >= amount) {
      balance -= amount;
      return true;
   }
   return false;
}
```

Key Features:

- Add money
- Deduct balance
- Transaction history

Purpose:

Provides an alternative payment system.

8.11. Ride Tracking Module

RideTracker.java

// Tracking + ETA

```
public void updateLocation(String location) {
   this.currentLocation = location;
}
public int calculateETA(int distance, int speed) {
   return distance / speed;
}
```

Key Features:

- Tracks ride progress
- Calculates ETA

Purpose:

Simulates real-time GPS tracking.

8.12. Notification Module

Notification.java

//basic communication system

```
public void sendNotification(String message) {
   System.out.println("Notification: " + message);
}
```

Key Features:

- Sends ride updates
- Promotional alerts
- Payment notifications

Purpose:

Maintains communication with users.

8.13. Rating & Review Module

RatingReview.java

Key Features:

- Submit reviews
- Calculate average rating

Purpose:

Improves service quality.

8.14. Promo Code Module

PromoCode.java

```
//discount logic
public double applyDiscount(double fare, double discount) {
   return fare - (fare * discount / 100);
}
```

Key Features:

- Discount application
- Usage validation

Purpose:

Enhances user engagement through offers.

8.15. SOS Emergency Module

SOSEmergency.java

```
// safety feature
public void triggerSOS(String location) {
   System.out.println("Emergency alert sent from: " + location);
}
```

Key Features:

- Emergency alerts
- Location sharing

Purpose:

Ensures passenger safety.

8.16. Admin Dashboard Module

AdminDashboard.java

```
// Admin functionality
public void generateReport(int totalRides, double revenue) {
   System.out.println("Total Rides: " + totalRides);
   System.out.println("Revenue: " + revenue);
}
```

Key Features:

- System monitoring
- Report generation

Purpose:

Provides control over the entire system.

8.17. GUI Implementation

GoProGUI.java

Features:

- Dashboard interface
- Ride booking interface
- User interaction panels

Purpose:

Provides a user-friendly graphical interface.

8.18. Integration of Modules

All modules are integrated in a sequence:

User → Ride Booking → Ride Matching → Fare Calculation → Payment → Tracking → Notification

This ensures a complete ride lifecycle.

8.19. Advantages of Implementation

- Modular and scalable design
- Real-world simulation
- Easy debugging and maintenance
- Clear separation of concerns

8.20. SUMMARY

This chapter discussed how each module presented the implementation details of the Go Pro Rental Vehicle Platform. It showed how each module is developed using Java and how these modules interact together to form a complete ride-hailing system.

The use of modular coding approach allows each module to perform a specific function while contributing to the overall ride-hailing application system. The integration of all modules results in a functional and efficient application that simulates real-world ride-hailing platforms.

CHAPTER 9: SYSTEM DESIGN

9.1. Introduction

The design of the software application system plays a great significance in improving the level of user interaction and usability. The Go Pro Rental Vehicle Platform consists of both a console-based application and a graphical user interface (GUI) developed through Java Swing.

In this chapter we explore the design of the software application system through various interface screens. These screenshots demonstrate how users, drivers, and administrators interact with the software system.

9.2. Overview of User Interface

The software application system consists of a user-friendly GUI that includes multiple panels including:

- Dashboard
- User Management
- Driver Management
- Vehicle Management
- Ride Booking
- Ride Tracking
- Wallet and Payment
- Admin Panel

The interface is designed to be intuitive, user-friendly, and responsive.

9.3. Dashboard Screen

Description:

The dashboard constitutes the home screen of the application. It highlights some of the system's statistics and provides the ability to execute quick access to major functionalities in one click.

Features:

- Displays total number of users, drivers, rides, and revenue
- Provides quick action buttons (Book Ride, Wallet, History)
- Navigation menu for accessing module applications

What to Capture:

- Full dashboard screen with statistics cards
- Sidebar menu

9.4. User Registration Screen

Description:

This screen facilitates new users' registration into the application system.

Features:

- Input fields for entering name, email address, mobile phone, and password
- Registration and verification button
- Table displaying registered users

What to Capture:

- User registration form
- User list table

9.5. Driver Management Screen

Description:

This screen is meant for handling information related to drivers.

Features:

- Driver registration form
- Driver verification
- Online/offline status update
- Driver list with details

What to Capture:

- Driver registration interface
- Driver list with status

9.6. Vehicle Management Screen

Description:

This screen is used for adding and managing vehicles.

Features:

- Vehicle type selection
- Input vehicle number, model, and colour
- Driver assignment
- Vehicle list display

What to Capture:

- Vehicle entry form
- Vehicle table

9.7. Ride Booking Screen

Description:

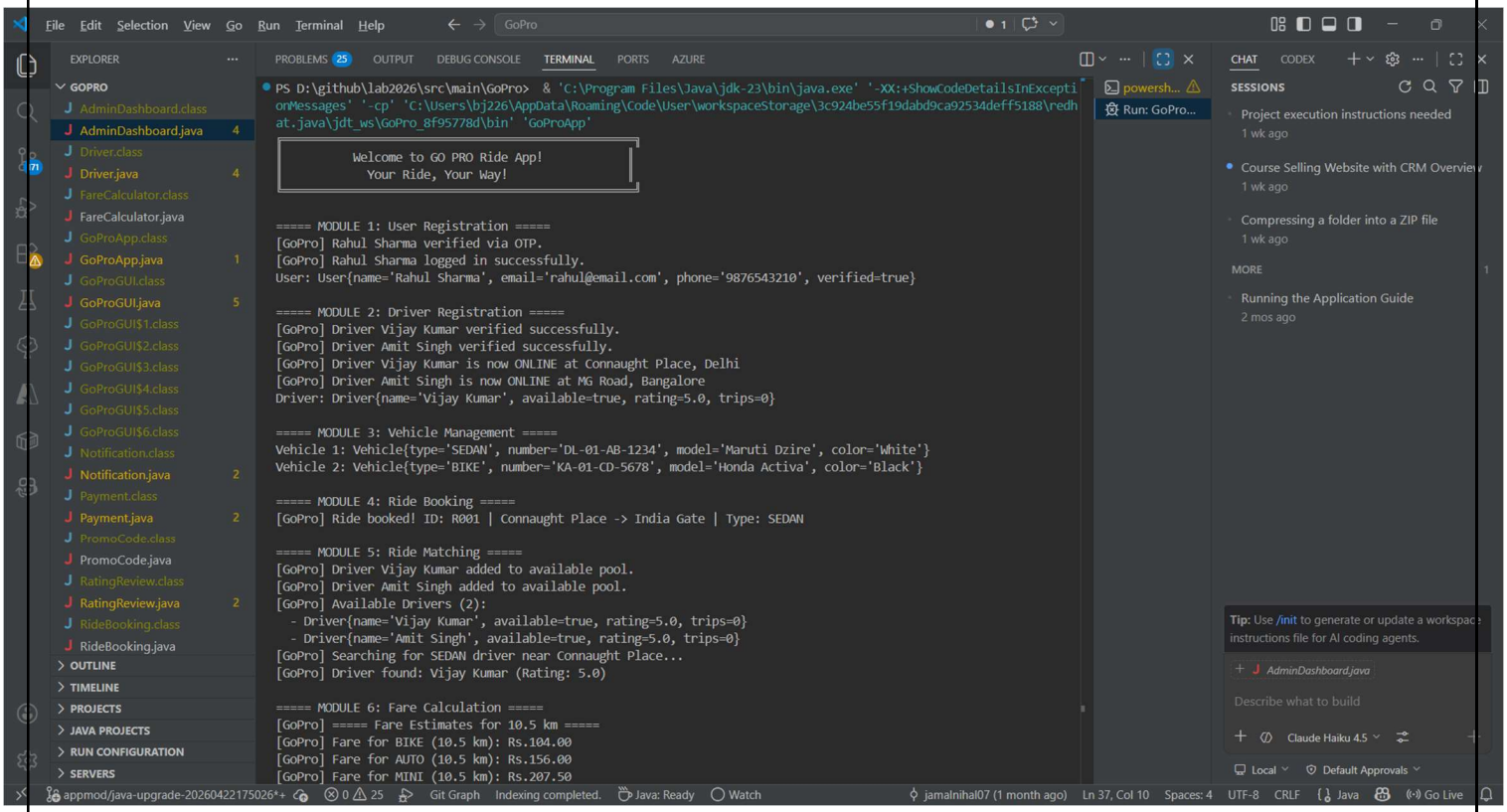
This screen allows users to book rides.

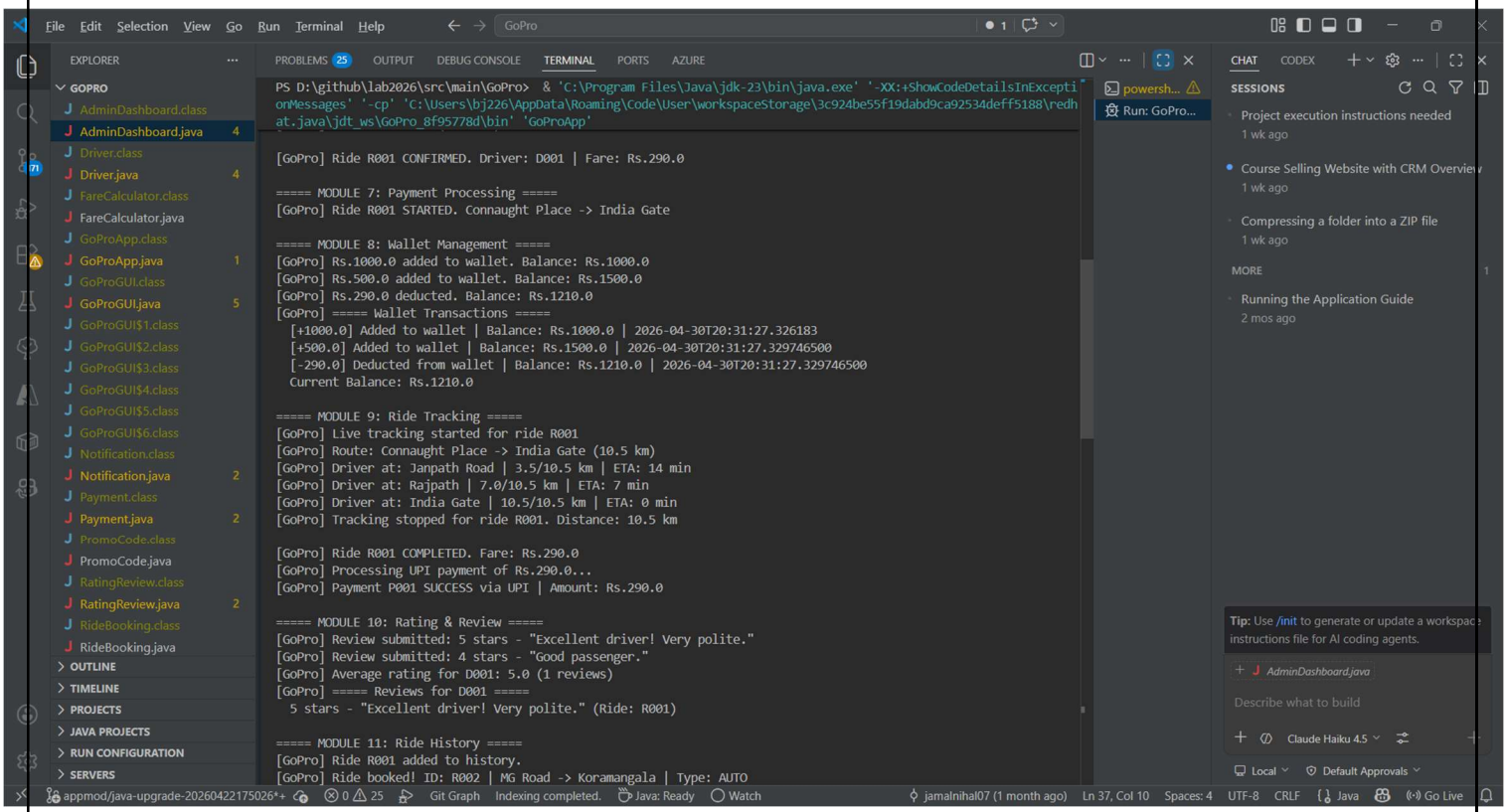
Features:

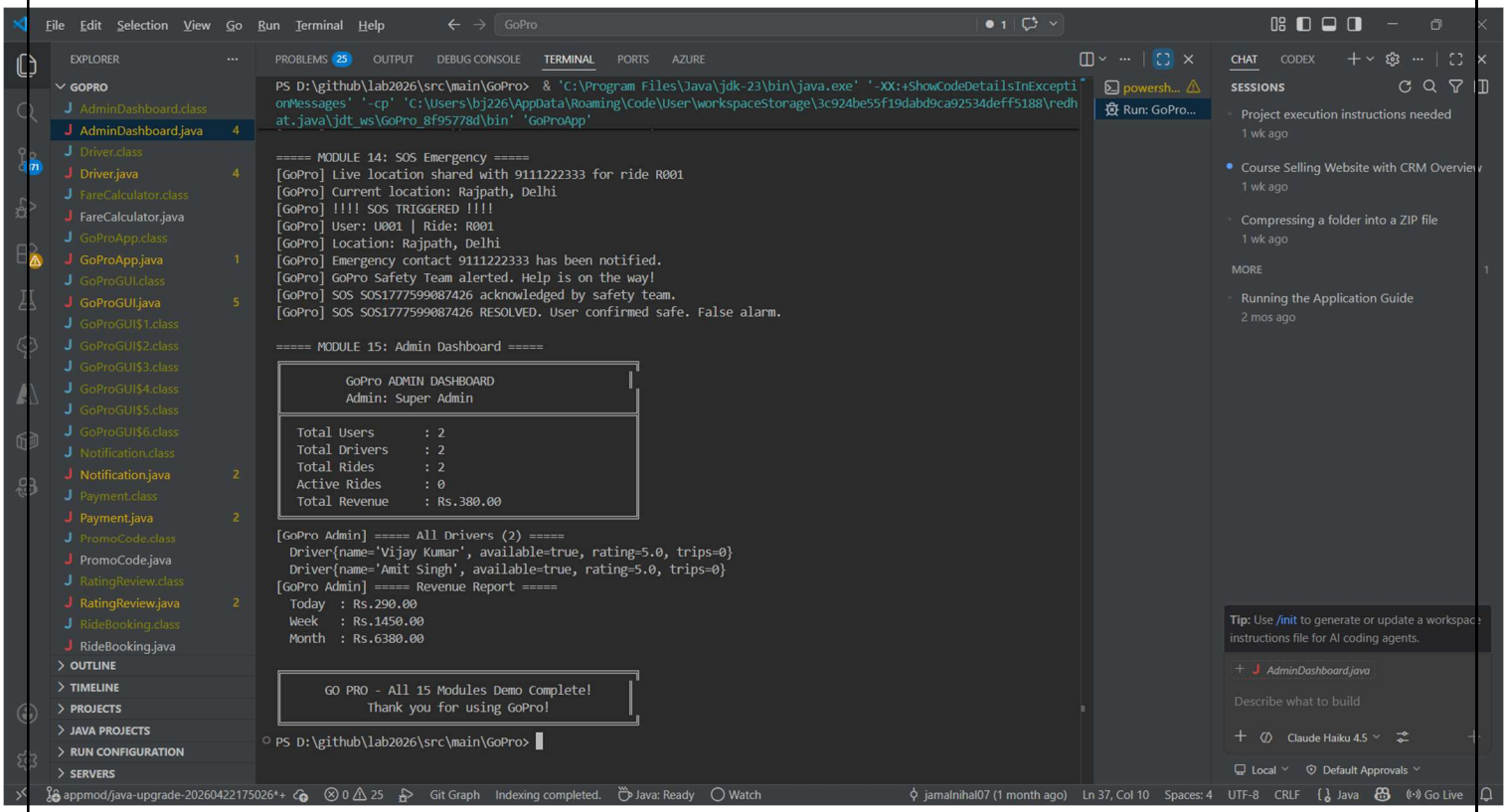
- User selection
- Distance input
- Ride type selection
- Fare estimation
- Booking confirmation
- Pickup point and destination input

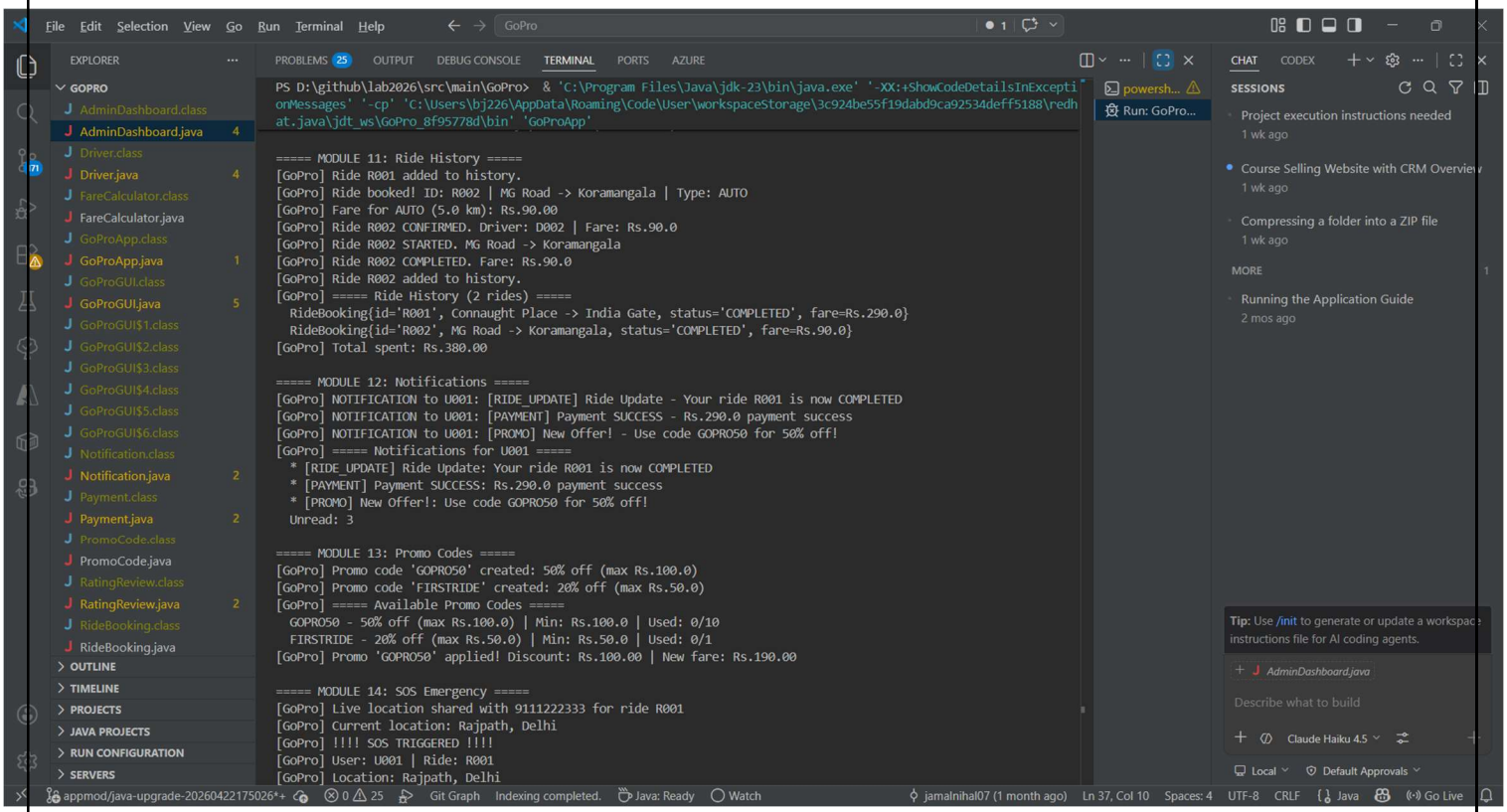
What to Capture:

- Ride booking form
- Fare estimation display


PS D:\github\lab2026\src\main\GoPro> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\bj226\AppData\Roaming\Code\User\workspaceStorage\3c924be55f19dabd9ca92534deff5188\redhat.java\jdt_ws\GoPro_8f95778d\bin' 'GoProApp'
Welcome to GO PRO Ride App!
Your Ride, Your Way!
===== MODULE 1: User Registration =====
[GoPro] Rahul Sharma verified via OTP.
[GoPro] Rahul Sharma logged in successfully.
User: User{name='Rahul Sharma', email='rahul@email.com', phone='9876543210', verified=true}
===== MODULE 2: Driver Registration =====
[GoPro] Driver Vijay Kumar verified successfully.
[GoPro] Driver Amit Singh verified successfully.
[GoPro] Driver Vijay Kumar is now ONLINE at Connaught Place, Delhi
[GoPro] Driver Amit Singh is now ONLINE at MG Road, Bangalore
Driver: Driver{name='Vijay Kumar', available=true, rating=5.0, trips=0}
===== MODULE 3: Vehicle Management =====
Vehicle 1: Vehicle{type='SEDAN', number='DL-01-AB-1234', model='Maruti Dzire', color='White'}
Vehicle 2: Vehicle{type='BIKE', number='KA-01-CD-5678', model='Honda Activa', color='Black'}
===== MODULE 4: Ride Booking =====
[GoPro] Ride booked! ID: R001 | Connaught Place -> India Gate | Type: SEDAN
===== MODULE 5: Ride Matching =====
[GoPro] Driver Vijay Kumar added to available pool.
[GoPro] Driver Amit Singh added to available pool.
[GoPro] Available Drivers (2):
- Driver{name='Vijay Kumar', available=true, rating=5.0, trips=0}
- Driver{name='Amit Singh', available=true, rating=5.0, trips=0}
[GoPro] Searching for SEDAN driver near Connaught Place...
[GoPro] Driver found: Vijay Kumar (Rating: 5.0)
===== MODULE 6: Fare Calculation =====
[GoPro] ===== Fare Estimates for 10.5 km =====
[GoPro] Fare for BIKE (10.5 km): Rs.104.00
[GoPro] Fare for AUTO (10.5 km): Rs.156.00
[GoPro] Fare for MINI (10.5 km): Rs.207.50


[GoPro] Ride R001 CONFIRMED. Driver: D001 | Fare: Rs.290.0
===== MODULE 7: Payment Processing =====
[GoPro] Ride R001 STARTED. Connaught Place -> India Gate
===== MODULE 8: Wallet Management =====
[GoPro] Rs.1000.0 added to wallet. Balance: Rs.1000.0
[GoPro] Rs.500.0 added to wallet. Balance: Rs.1500.0
[GoPro] Rs.290.0 deducted. Balance: Rs.1210.0
[GoPro] ===== Wallet Transactions =====
[+1000.0] Added to wallet | Balance: Rs.1000.0 | 2026-04-30T20:31:27.326183
[+500.0] Added to wallet | Balance: Rs.1500.0 | 2026-04-30T20:31:27.329746500
[-290.0] Deducted from wallet | Balance: Rs.1210.0 | 2026-04-30T20:31:27.329746500
Current Balance: Rs.1210.0
===== MODULE 9: Ride Tracking =====
[GoPro] Live tracking started for ride R001
[GoPro] Route: Connaught Place -> India Gate (10.5 km)
[GoPro] Driver at: Janpath Road | 3.5/10.5 km | ETA: 14 min
[GoPro] Driver at: Rajpath | 7.0/10.5 km | ETA: 7 min
[GoPro] Driver at: India Gate | 10.5/10.5 km | ETA: 0 min
[GoPro] Tracking stopped for ride R001. Distance: 10.5 km
[GoPro] Ride R001 COMPLETED. Fare: Rs.290.0
[GoPro] Processing UPI payment of Rs.290.0...
[GoPro] Payment P001 SUCCESS via UPI | Amount: Rs.290.0
===== MODULE 10: Rating & Review =====
[GoPro] Review submitted: 5 stars - "Excellent driver! Very polite."
[GoPro] Review submitted: 4 stars - "Good passenger."
[GoPro] Average rating for D001: 5.0 (1 reviews)
[GoPro] ===== Reviews for D001 =====
5 stars - "Excellent driver! Very polite." (Ride: R001)
===== MODULE 11: Ride History =====
[GoPro] Ride R001 added to history.
[GoPro] Ride booked! ID: R002 | MG Road -> Koramangala | Type: AUTO





9.8. Ride Tracking Screen

Description:

This screen shows ride progress and driver's current location.

Features:

- Driver location updates
- Distance covered
- Time Remaining/Estimated time of arrival (ETA)

What to Capture:

- Tracking ride interface showing progress

9.9. Wallet & Payment Screen

Description:

This screen deals with financial transactions.

Features:

- Add money into wallet
- Payment processing
- View balance
- Transaction history

What to Capture:

- Wallet interface
- Payment confirmation

9.10. Rating & Review Screen

Description:

This screen allows users to rate drivers and provide feedback comments about them.

Features:

- Rating input (1–5 stars)
- Display of previous reviews
- Comment section

What to Capture:

- Rating submission interface

9.11. Notification Screen

Description:

Displays various system notifications and alerts.

Features:

- Ride notification updates
- Promotional messages
- Payment notifications

What to Capture:

- Notification list

9.12. Promo Code Screen

Description:

This screen allows applying promo codes to one's account.

Features:

- Enter promo code
- Discount calculation
- Display list of valid offers available

What to Capture:

- Promo code application

9.13. SOS Emergency Screen

Description:

Provides emergency support features.

Features:

- SOS button
- Emergency contact notification
- Location sharing
-

What to Capture:

- SOS trigger interface

9.14. Admin Dashboard Screen

Description:

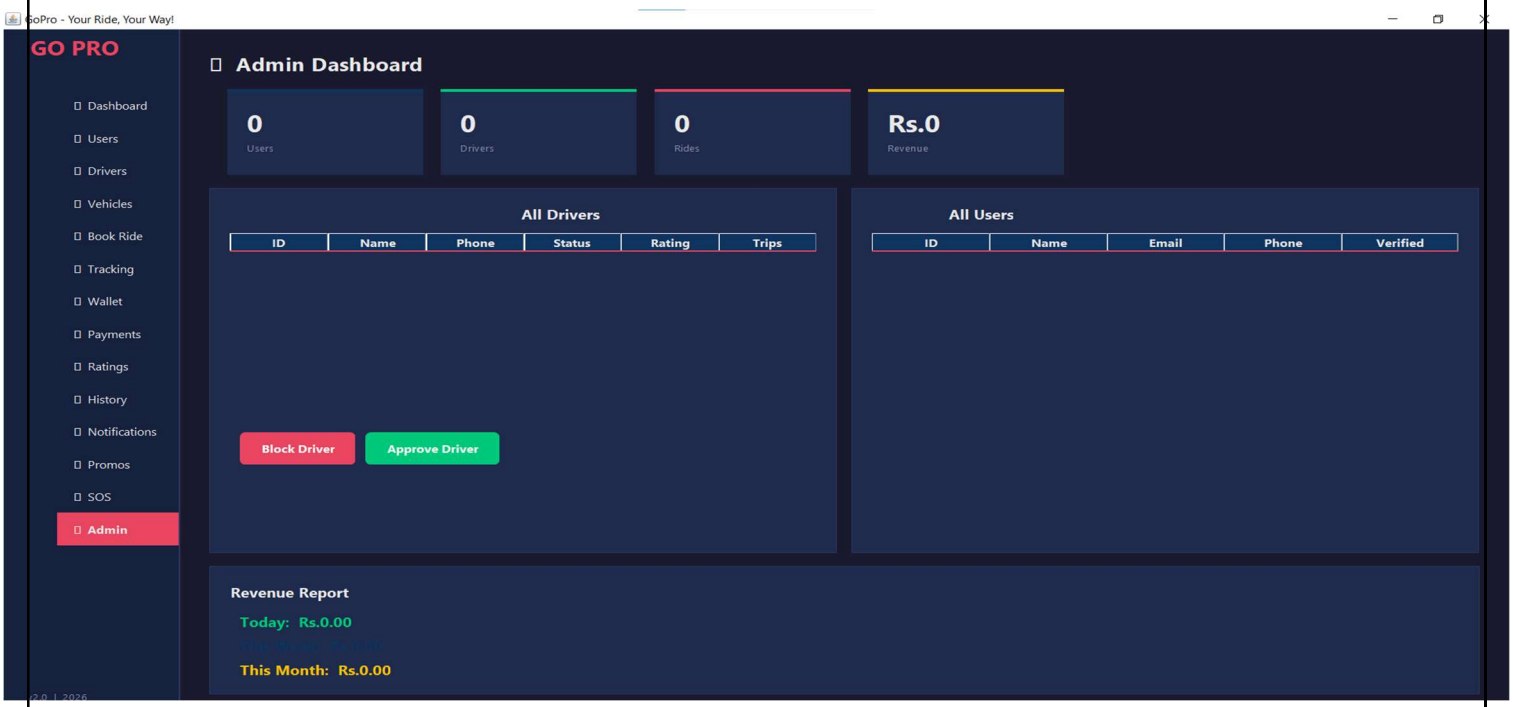
The admin panel provides control over the entire system.

Features:

- System statistics
- User and driver management
- Revenue/financial reports

What to Capture:

- Administrator dashboard with statistics



9.15. Design Characteristics

The system design follows these principles:

- **User-Friendly Interface:** Easy navigation and simple layout
- **Consistency:** Uniform design across all sections
- **Responsiveness:** Quick interaction to user interactions with minimal delay
- **Clarity:** Clear labels and structured forms

9.16. SUMMARY

This chapter presented the system design of the Go Pro Rental Vehicle Platform through various interfaces of the screens. Each screen was described in terms of its functionality and features, providing a clear understanding of how users interact with the system.

The graphical interface enhances usability as it shows how the system is put into use displaying the practical implementation of the system. These designs are important aspects playing a crucial role in making the application user-friendly and effective.

CHAPTER 10: TESTING

10.1. Introduction

Testing is a crucial phase in the Software Development Life Cycle (SDLC) that ensures the correctness, reliability, and performance of the system. It involves verifying that the system meets the specified requirements and functions as expected under different conditions.

In the Go Pro Rental Vehicle Platform, testing is performed to validate all modules such as user management, ride booking, payment processing, and tracking. The goal is to identify and fix errors, ensure system stability, and improve overall performance.

10.2. Objectives of Testing

The main objectives of testing are:

- To verify that all functionalities work correctly
- To detect and remove errors or bugs
- To ensure system reliability and performance
- To validate user requirements
- To ensure smooth interaction between modules

10.3. Types of Testing Performed

10.3.1. Unit Testing

Unit testing focuses on testing individual modules or components of the system.

Examples:

- Testing user login functionality
- Testing payment processing
- Testing fare calculation logic

Outcome:
Each module was tested independently and found to be functioning correctly.

10.3.2. Integration Testing

Integration testing ensures that different modules work together properly.

Examples:

- Ride booking + driver matching
- Fare calculation + payment processing
- Ride tracking + notification system

Outcome:
Modules were successfully integrated without major issues.

10.3.3. System Testing

System testing evaluates the complete system as a whole.

Examples:

- End-to-end ride booking process
- Complete payment workflow
- Full system execution from start to finish

Outcome:
The system performed as expected under normal conditions.

10.3.4. User Acceptance Testing (UAT)

This testing is performed to ensure that the system meets user expectations.

Examples:

- User interface usability
- Ease of navigation
- Response time

Outcome:
The system was found to be user-friendly and easy to operate.

10.3.5. Performance Testing

Performance testing checks system speed and responsiveness.

Examples:

- Response time for ride booking
- Processing time for payments

Outcome:
The system responded efficiently with minimal delay.

10.4. Test Case Design

Test cases are used to validate system functionality.

Test Case 1: User Registration

Parameter	Description
Test Case ID	TC01
Objective	Verify user registration
Input	Valid user details
Expected Output	User registered successfully
Actual Output	User registered successfully
Status	Pass

Test Case 2: Driver Verification

Parameter	Description
Test Case ID	TC02
Objective	Verify driver verification
Input	Valid license number
Expected Output	Driver verified
Actual Output	Driver verified
Status	Pass

```
===== MODULE 2: Driver Registration =====
[GoPro] Driver Vijay Kumar verified successfully.
[GoPro] Driver Amit Singh verified successfully.
[GoPro] Driver Vijay Kumar is now ONLINE at Connaught Place, Delhi
[GoPro] Driver Amit Singh is now ONLINE at MG Road, Bangalore
Driver: Driver{name='Vijay Kumar', available=true, rating=5.0, trips=0}
```

Test Case 3: Ride Booking

Parameter	Description
Test Case ID	TC03
Objective	Verify ride booking
Input	Pickup, destination, ride type
Expected Output	Ride created successfully
Actual Output	Ride created successfully
Status	Pass

```
===== MODULE 4: Ride Booking =====
[GoPro] Ride booked! ID: R001 | Connaught Place -> India Gate | Type: SEDAN
```

Test Case 4: Fare Calculation

Parameter	Description
Test Case ID	TC04
Objective	Verify fare calculation
Input	Distance and ride type
Expected Output	Correct fare displayed
Actual Output	Correct fare displayed
Status	Pass

```
===== MODULE 6: Fare Calculation =====
[GoPro] ===== Fare Estimates for 10.5 km =====
[GoPro] Fare for BIKE (10.5 km): Rs.104.00
[GoPro] Fare for AUTO (10.5 km): Rs.156.00
[GoPro] Fare for MINI (10.5 km): Rs.207.50
[GoPro] Fare for SEDAN (10.5 km): Rs.290.00
[GoPro] Fare for SUV (10.5 km): Rs.382.50
[GoPro] Fare for SEDAN (10.5 km): Rs.290.00
```

Test Case 5: Payment Processing

Parameter	Description
Test Case ID	TC05
Objective	Verify payment processing
Input	Payment details
Expected Output	Payment successful
Actual Output	Payment successful

Parameter	Description
Status	Pass

Test Case 6: Ride Tracking

Parameter	Description
Test Case ID	TC06
Objective	Verify ride tracking
Input	Ride in progress
Expected Output	Location updates displayed
Actual Output	Location updates displayed
Status	Pass

Test Case 7: SOS Emergency

Parameter	Description
Test Case ID	TC07
Objective	Verify SOS functionality
Input	Trigger SOS
Expected Output	Emergency alert sent
Actual Output	Emergency alert sent
Status	Pass

```
===== MODULE 14: SOS Emergency =====
[GoPro] Live location shared with 9111222333 for ride R001
[GoPro] Current location: Rajpath, Delhi
[GoPro] !!!! SOS TRIGGERED !!!!
[GoPro] User: U001 | Ride: R001
[GoPro] Location: Rajpath, Delhi
[GoPro] Emergency contact 9111222333 has been notified.
[GoPro] GoPro Safety Team alerted. Help is on the way!
[GoPro] SOS SOS1777827560711 acknowledged by safety team.
[GoPro] SOS SOS1777827560711 RESOLVED. User confirmed safe. False alarm.
```

10.5. Error Handling and Debugging

During testing, minor issues were identified and resolved:

- Input validation errors were corrected
- Null value handling was improved
- System messages were refined for clarity

Debugging was performed using IDE tools to trace and fix issues efficiently.

10.6. Testing Tools Used

- Java IDE (IntelliJ / Eclipse / NetBeans)
- Console output for debugging
- Manual testing techniques

10.7. Test Results

All test cases were executed successfully, and the system performed as expected. No major defects were found after testing.

10.8. SUMMARY

This chapter discussed the testing process of the Go Pro Rental Vehicle Platform. Various testing methods such as unit testing, integration testing, system testing, and user acceptance testing were performed to ensure system reliability and correctness.

Test cases were designed and executed to validate system functionality, and all results were successful. The testing process confirmed that the system meets the required specifications and performs efficiently.

CHAPTER 11: CONCLUSION

11.1. Introduction

The conclusion is a vital section that should be included in any project report as it provides information about the summary of the entire project undertaken, its key success factors, and also the effectiveness of the developed system. The Go Pro Rental Vehicle Platform was designed and implemented to simulate a real-world ride-hailing application using modern programming concepts and modular design.

The following chapter presents the overall outcomes of the project, which is a summary of the success achieved during the development of the project.

11.2. Project Outcome

The Go Pro system successfully demonstrates the working of a ride-hailing platform similar to modern applications such as Ola and Uber. The system integrates multiple modules, including user management, driver management, ride booking, fare calculation, payment processing, and ride tracking.

The project achieves the following:

- Implementation of a complete ride lifecycle from booking to payment
- Integration of multiple independent modules into a single system
- Simulation of real-time features such as tracking and notifications
- Development of a user-friendly graphical interface
- Application of object-oriented programming concepts

The system effectively replicates the core functionalities of a real-world transportation platform.

11.3. Achievement of Objectives

All objectives defined in Chapter 2 have been successfully achieved:

- A functional ride-hailing system has been developed
- Real-time ride booking and driver matching have been implemented
- Dynamic fare calculation has been successfully integrated
- Payment processing and wallet system are operational
- Safety features such as SOS emergency have been included
- The system follows a modular and scalable architecture

This confirms that the project meets both functional and technical goals.

11.4. Strengths of the System

The developed system has several strengths:

- **Modular Design:** Each component is independent and easy to maintain
- **User-Friendly Interface:** The GUI is intuitive and easy to use

- **Comprehensive Functionality:** Covers all major features of a ride-hailing system
- **Scalability:** Can be extended for real-world deployment
- **Simulation of Real Systems:** Closely resembles actual ride-hailing platforms

These strengths make the system efficient and reliable.

11.5. Limitations of the System

Despite its advantages, the system has certain limitations:

- Real-time GPS tracking is simulated
- Payment processing is not integrated with actual gateways
- The system is not deployed on a cloud platform
- Limited scalability due to local execution
- No real database connectivity

These limitations are primarily due to the academic nature of the project.

11.6. Learning Outcomes

The creation and development of this project have been very beneficial in terms of learning experiences:

- Understanding of software development lifecycle (SDLC)
- Practical application of Object-Oriented Programming concepts
- Experience in designing modular systems
- Knowledge of GUI development using Java Swing
- Exposure to real-world system design and problem-solving

11.7. Final Remarks

The Go Pro Rental Vehicle Platform can be used as an excellent starting point for understanding the design and construction of ride-hailing systems in today's world. The platform illustrates the use of different tools and concepts in constructing a highly efficient system.

With further enhancements, this project can be extended into a real-world application.

11.8. SUMMARY

This chapter summarized the overall outcomes of the Go Pro Rental Vehicle Platform. It highlighted the successful implementation of system functionalities, achievement of objectives, strengths, and limitations of the system. The project demonstrates a clear understanding of modern software development practices and real-world system design.

CHAPTER 12: FUTURE SCOPE

12.1. Introduction

All software application systems have the potential scope for further enhancement and expansion. Although the Go Pro Rental Vehicle Platform successfully achieves all the core functionalities of a ride-hailing software system, there are several improvement opportunities and extensions can be made through advanced technologies and real-world implementations.

This chapter outlines various possible improvements that can transform the current system into a fully practical ride-hailing application on an industry-level.

12.2. Integration with Real-Time GPS

While the current system employs location tracking simulation techniques, real-time GPS integration can be considered in future versions. This can be accomplished through the use of mapping tools.

Enhancements:

- Real-time GPS integration with Google Maps API
- Real-time driver location tracking
- Route optimization and navigation support
- Accurate ETA based on route calculation

This will significantly improve reality of system and usability.

12.3. Online Payment Gateway Integration

While the current system uses simulated online payment processing capabilities, these can be enhanced by integrating real payment gateways.

Enhancements:

- Integration with UPI, Razorpay, Paytm, and Stripe payment gateways
- Secure online transactions
- Automatic billing and invoicing facility
- Refund management

This will make the system suitable for real-world deployment.

12.4. Database Integration

Currently, the system uses in-memory data storage. Future versions may feature database connectivity.

Enhancements:

- Integration with MySQL or MongoDB
- Persistent data storage
- Improved data management
- Support for large-scale data

These upgrades will ensure better scalability and reliability.

12.5. Mobile Application Development

The system can be transformed into a mobile application for better accessibility.

Enhancements:

- Android/iOS app development
- Location-based services
- Real-time notifications
- Improved user experience

These improvements will make the platform more user-friendly and widely accessible.

12.6. Cloud Deployment

At the moment, the system currently runs locally. Future deployment can be hosted in the cloud platforms for scalability and better performance.

Enhancements:

- Deployment on AWS, Azure, or Google Cloud services
- Load balancing and distributed computing
- High availability and fault tolerance

12.7. Artificial Intelligence Integration

The implementation of AI technologies will help to improve system efficiency, optimize the process, and decision-making.

Enhancements:

- Smart driver allocation using AI algorithms
- Demand prediction and surge pricing optimization
- Sending personalized user recommendations

12.8. Enhanced Security Features

More security features can be further implemented to protect user data and transactions.

Enhancements:

- Data encryption
- Secure APIs
- Multi-factor authentication
- Fraud detection algorithm systems

12.9. Carpooling and Ride Sharing

The system can be expanded to support services like shared ride facility.

Enhancements:

- Support of multiple passengers in a single ride
- Cost sharing facility
- Route optimization

This enhancement will reduce travel cost and traffic congestion.

12.10. Electric Vehicle Integration

Future versions of platform can support eco-friendly transportation vehicle.

Enhancements:

- Integration of electric vehicles (EVs)
- Mapping of charging stations
- Green ride options

12.11. Voice-Based Booking System

Advanced user interaction mode can be implemented using voice recognition.

Enhancements:

- Voice commands for booking rides
- AI-based assistant integration
- Accessible to differently-abled users

12.12. Advanced Analytics Dashboard

The admin dashboard panel can be enhanced with analytics capability.

Enhancements:

- Real-time reports
- Revenue/sales analysis
- User behaviour tracking
- Predictive analytics

12.13. IoT Integration

Internet of Things (IoT) can be used to improve monitoring and safety measures.

Enhancements:

- Fitment of GPS trackers on vehicles
- Driver behaviour monitoring
- Sending of safety alerts

12.14. SUMMARY

The future scope of Go Pro Rental Vehicle Platform was highlighted in the present chapter and various enhancements were discussed which can increase the functionality, scalability and usability of the system. The important enhancements include integration with real-time GPS support, payment gateways, databases, mobile applications, and cloud platforms.

Advanced technologies such as artificial intelligence, IoT, and analytics can further enhance the system and make it suitable for real-world deployment. Overall, the project has immense potential in terms of commercialization of the product.

CHAPTER 13: BIBLIOGRAPHY

13.1. Introduction

Bibliography provides the list of sources and references used during the development of the project. These sources include research papers, technical documentation, books, and online resources that helped in understanding and implementing the Go Pro Rental Vehicle Platform.

All references listed below are recent (post-2020) and relevant to ride-hailing systems, software development, and modern technologies.

13.2. References

1. Singh, A., & Kumar, R. (2023). *Design and Development of Ride-Hailing Systems Using Modern Technologies*. International Journal of Computer Applications.
2. Sharma, P., & Verma, S. (2022). *A Study on Smart Transportation Systems and Ride-Sharing Applications*. IEEE Xplore Digital Library.
3. Gupta, V., & Mehta, D. (2024). *Efficient Ride Allocation Algorithms in Ride-Hailing Platforms*. Springer Journal of Intelligent Systems.
4. ResearchGate (2025). *GoRide: A Modern Ride-Hailing App Solution*. Available online.
5. Oracle Corporation (2023). *Java SE Documentation*. Available at: https://docs.oracle.com/
6. Oracle Corporation (2022). *Java Swing Tutorial*. Available at: https://docs.oracle.com/javase/tutorial/uiswing/
7. Google Developers (2024). *Google Maps Platform Documentation*. Available at: https://developers.google.com/maps
8. Amazon Web Services (2023). *Cloud Computing Concepts and Services*. Available at: https://aws.amazon.com/
9. Microsoft Azure (2023). *Cloud Application Development Guide*. Available at: https://azure.microsoft.com/
10. MongoDB Inc. (2024). *MongoDB Documentation*. Available at: https://www.mongodb.com/docs/
11. MySQL Documentation Team (2023). *MySQL Reference Manual*. Available at: https://dev.mysql.com/doc/
12. GeeksforGeeks (2023). *Object-Oriented Programming Concepts in Java*. Available at: https://www.geeksforgeeks.org/
13. TutorialsPoint (2022). *Java Programming and Swing GUI*. Available at: https://www.tutorialspoint.com/
14. IEEE (2021). *Intelligent Transport Systems and Smart Mobility Solutions*. IEEE Journals.
15. NITI Aayog (2021). *Future of Mobility in India: Ride-Sharing and Digital Transport*. Government of India Report.